arXiv:2306.06823v1 [cs.CV] 12 Jun 2023

Weakly supervised information extraction from inscrutable handwritten document images

Sujoy Paul, Gagan Madan, Akankshya Mishra, Narayan Hegde, Pradeep Kumar, and Gaurav Aggarwal

Google Research

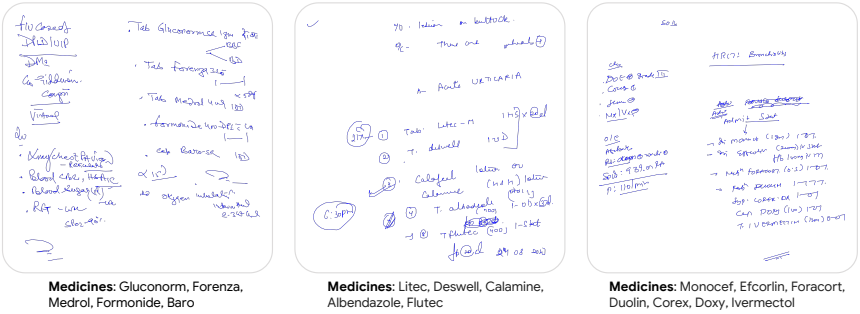
Abstract. State-of-the-art information extraction methods are limited by OCR errors. They work well for printed text in form-like documents, but unstructured, handwritten documents still remain a challenge. Adapting existing models to domain-specific training data is quite expensive, because of two factors, 1) limited availability of the domain-specific documents (such as handwritten prescriptions, lab notes, etc.), and 2) annotations become even more challenging as one needs domain-specific knowledge to decode inscrutable handwritten document images. In this work, we focus on the complex problem of extracting medicine names from handwritten prescriptions using only weakly labeled data. The data consists of images along with the list of medicine names in it, but not their location in the image. We solve the problem by first identifying the regions of interest, i.e., medicine lines from just weak labels and then injecting a domain-specific medicine language model learned using only synthetically generated data. Compared to off-the-shelf state-of-the-art methods, our approach performs $> 2.5\times$ better in medicine names extraction from prescriptions.

Keywords: handwriting · language model · prescription · weakly-supervised

1 Introduction

Optical character recognition (OCR) enables the translation of any image containing text into analyzable, editable and searchable format. Over the last decade, many large scale models [10, 18, 26] and sophisticated techniques [4, 5, 29] have been developed with neural network based architectures for OCR. These systems are not only limited to printed text but also work quite well on handwritten text, as they are trained on large amount of labeled as well as synthetic handwritten data. In the past, there have also been works around developing domain specific OCR models [6, 21, 41]. Most of these works develop these models for generic text lines [20, 31], and require meticulously labeled data for learning. In this work, we primarily focus on how we can improve the quality of existing OCR models on very hard to read, unstructured documents for specific entities of interest, with an application in handwritten medical prescriptions.

In many countries, prescriptions are primarily delivered to patients in handwritten formats by doctors. A few billion prescriptions are generated every year



Medicines: Gluconorm, Forenza, Medrol, Formonide, Baro

Medicines: Litec, Deswell, Calamine, Albendazole, Flutec

Medicines: Monocef, Efcorlin, Foracort, Duolin, Corex, Doxy, Ivermectol

Fig. 1: Samples representative images from the prescription dataset used in this work. As we can see the handwriting is often inscrutable and does not follow any specific structure or format. The task we focus in this paper is to extract medicine names from such images.

world-wide [19]. Digitizing them would unlock numerous applications for many stakeholders and use cases in the healthcare ecosystem like e-pharmacies, insurance companies, creating electronic health records necessary for preventive healthcare, better diagnosis, analysis at a local and global level for policy making and so on. However, most of such documents, as shown in Figure 1 are often hard to read for non-pharmacists [33]. Even pharmacists go through months/years of training to decipher such prescriptions. Existing state-of-the-art OCR models though trained on large amount of data, do not perform well on such inscrutable documents. Procuring large domain-specific datasets is not a cost-effective or scalable solution, as it involves annotation that too from domain experts which can become quite expensive. Although there have been some works [1, 15, 34] in extracting information from handwritten prescriptions, the algorithms are not generalizable, heavily hand-tuned and lack rigorous evaluations. With these problems in mind, we propose an approach that can significantly enhance the performance of existing state-of-the-art OCR systems by selectively infusing domain knowledge using only weak supervision.

Medical prescriptions consist of various information like data from lab reports, ordered tests, health vitals, observations along with medicine names. Our work focuses on the medicine section which is considered the most important from a consumer standpoint, but the techniques can be similarly applied to other sections or other types of documents beyond prescriptions, such as printed forms filled with handwriting. The medicine section of a prescription has a rough semantics consisting of medicine name, category, frequency of intakes and quantity (see Figure 1). As these are non-form type of documents and quite unstructured, it is a challenge to extract medicine name entities from such documents.

Most OCR approaches [18, 26] take a two step approach - first localize the text regions by detecting bounding boxes around them, and then recognizing the text using line recognition models. The recognition model often consists of an optical recognizer and a language model (LM) to correct the optical model errors. The

LM gives us the flexibility to infuse domain-specific knowledge. But, injecting such knowledge to all lines in the document may not be optimal, as different parts of the document can correspond to different entities, or even domains. For example, the pattern in which a medicine name is written is very different from the pattern in which normal text such as observations are written in the same prescription. Thus, in order to enhance the recognition of medicine names and extract them from the prescription, we first detect lines where medicine names are written. Then in the recognition model, we inject a LM which is specific to medicine names. For the rest of the image, we inject the vanilla LM.

Note that to learn the model which detects medicine lines, we do not use strong bounding polygon labels, but rather only weak labels, i.e., the medicine names present in the image. Such weak labels are much easier to obtain, as the annotators do not need to draw a bounding polygon and often labeling comes for free, for example, when a medicine bill is paired with a prescription. Apart from that, to learn the medicine LM, we do not use any annotated text lines, but rather generate synthetic text lines using a probabilistic programming approach. Our weakly supervised medicine line detector obtains 78% pixel mIoU with just weak labels, and helps to selectively infuse medicine LM, which in turn improves the overall performance from 19% to 48% jaccard index. The main contributions of this work are:

- Develop a weakly supervised segmentation method to detect specific text entities, such as medicine names in handwritten prescriptions.
- Learning a domain-specific medicine LM using synthetic medicine name lines generated by probabilistic programs and using it to enhance the performance of state-of-the-art OCR models.
- A model dependent unique way of enhancing the performance of matching with words from the vocabulary.

2 Related Works

Optical Character Recognition OCR literature has seen tremendous improvements in the past decade. The successes [10, 18, 26] can be attributed to sophisticated models, synthetic data generation, various augmentation techniques, among others. An OCR system is made of multiple models, starting from text detection [29, 30, 43], script identification [12, 17], and finally line recognition [3, 10, 26, 27]. Even with all these advancements, recognition of handwritten lines still remains a challenging task as writing style can be a unique signature of the person, allowing room for huge variations. In our experiments, we found that off-the-shelf line recognition models, even though perform quite well for a lot of printed and handwritten datasets, they fail to perform equally well on handwritten images. In this work, we show how we can improve their accuracy by more than 2 times the baseline by first detecting specific entities of interest (rather than detecting all text) and then improving the line recognition model by injecting domain-specific LMs. We next discuss the existing literature around these topics.

Weakly-supervised Detection Detecting specific entities of interest in an image can be posed as detection or segmentation task. However, to learn these tasks, traditional methods would need strong labels, i.e., either pixel-wise [9,30] or bounding box labels [30,37,38]. In the recent past, there has been a lot of work in developing methods which can learn from only weak labels, such as weakly-supervised object detection [25,47], segmentation [22,44], action detection [32,46], etc. These methods do not need access to strong labels such as bounding boxes, but can learn from just weak labels, i.e., image-level labels of the object categories present in the individual training images. Such a formulation reduces the manual labor needed to acquire strong labels, thus making it scalable to large datasets.

Motivated by these, we aim to learn a segmentation model to detect entities of interest in an image, such as medicine names from just weak labels, i.e., list of medicine names given an image. In this use case, the individual entities do not correspond to any underlying category unlike segmentation or detection of objects in natural scenes. Recently, it has been shown [23] that using weakly labeled data along with strong labels improves the performance of scene text recognition. In our task, we only have weakly labeled data without any strong labels (synthetic or real) and the text is primarily handwritten which is often inscrutable even if text detection is perfectly done. Moreover in our use case, we need to detect specific entities among other cluttered text, and not any generic text. There are also works on defining rules to derive weak labels from the data [36]. While that is quite challenging and not generalizable in our use case, we use the intuition to convert the weak labels to strong labels via labeling functions.

Domain-specific Language Models There has been a lot of work [24,35,45] which shows that injecting domain-specific knowledge in LMs helps to perform much better on those domains than models developed on generic text. Specifically for OCR, there have been some works [11,14] showing that having access to domain related text data helps to adapt existing LMs and thus improving final OCR performance. However, in our use case of decoding medicine names, it is non-trivial to acquire lines of medicine names written by doctors, as they are hardly available in normal text corpus. To solve that, we use domain knowledge to define a probabilistic program which can take in the medicine name and generates patterns of medicine lines as would be written by doctors in prescriptions. We show that using such a LM in the OCR decoder improves the performance significantly.

3 Methodology

3.1 Problem Statement

In this work, we focus on the problem statement of extracting textual entities from non-form type handwritten document images, which are often hard to read. We specifically focus on the challenging problem of extracting medicine names from handwritten prescriptions as shown in Figure 1. Formally, given an image $\boldsymbol{x}$,

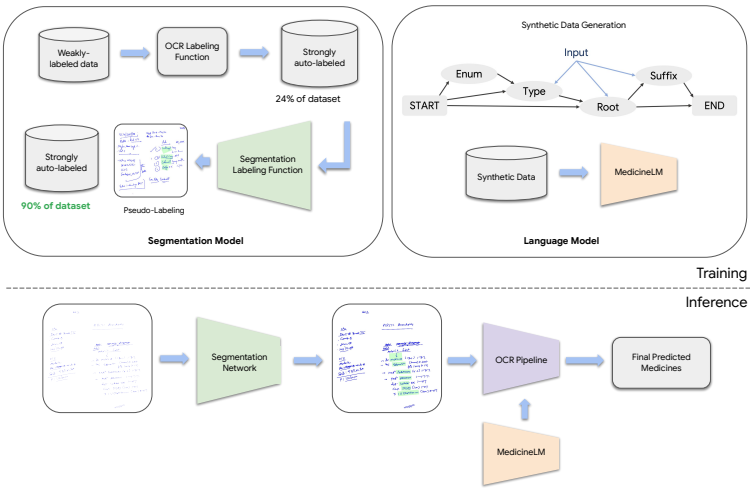


Fig. 2: Training and inference pipelines for medicine name extraction from prescriptions. The top-left block shows the weakly supervised medicine line segmentation pipeline. The top-right block shows the process of generating synthetic medicine lines using probabilistic programs and then using it to train a medicine LM. The bottom row shows the inference pipeline, that first localizes the medicine names using the segmentation network, and then injects the medicine LM while decoding the OCR outputs.

the output of the framework should be the medicine names $\{m_j\}_{j=1}^{n}$ that appear in the image, where $m_j \in \mathrm{V}$, the vocabulary of medicines. n denotes the number of medicines in the prescription that varies from prescription to prescription. The training data that we use to solve this problem is only weakly labeled, i.e. for every image, we have a list of medicine names that appear in the image, and not their bounding box locations. Thus, our training data contains tuples of image and unordered set of medicine names as follows, $\mathcal{D} = \{(\boldsymbol{x}_i, \mathcal{G}_i = \{m_j\}_{j=1}^{n_i})\}_{i=1}^{N}$, where n_i denotes the number of medicines in that image, N denotes the number of images in the training data and $\mathcal{G}_i$ is the ground truth list of medicines.

3.2 OCR Line Recognition Model

Most line recognition models have two parts - the encoder, often called the optical part of the model, which encodes the visual information, and the decoder, which is either trained end-to-end with the encoder, or CTC type decoder [13] where the encoder outputs are combined with LM scores to obtain the final text. We use the second option and train our network with CTC loss [13]. This allows us to decouple the optical and the LM, and replace it with domain specific LMs.

Encoder: The encoder or the optical part of the line recognizer consists of first 7 layers of inverted bottleneck conv layers [39] with 64 filters and stride of 1, followed by 12 layers of transformer encoder [42] with hidden size of 256 and 4

attention heads, and finally a fully connected symbol classification head. We use this backbone from [10], as it achieves state-of-the-art performance on various datasets. Our pre-trained model is also the same as [10]. It is interesting to note that our method is agnostic to the encoder used as it can be used to boost the performance of any OCR backbone.

Decoder: We use a CTC decoder [13] following [10], which combines scores from the encoder logits and a character n-gram LM. We set $n = 9$ unless otherwise mentioned. We will discuss how we train and use a medicine LM subsequently.

3.3 Weakly Supervised Line Segmentation

We next discuss our algorithm to detect medicine lines by just using weak labels while training, i.e., only the medicine names for every image, and not their bounding polygons. Note that while we use this method for medicine line detection, it can be also used for detecting other entities in other document types.

Labeling Functions At the core of our algorithm is the idea of using labeling functions to automatically convert a weakly labeled dataset to strongly labeled. There have been some works [36] in literature where rules are defined as labeling functions. The labeling functions may not be as perfect as a human oracle and the strong labels they generate may have errors in them. There are often thresholds or rules used to reduce errors. Thus, while defining a labeling function we need to optimize coverage, which is the number of data points that can be labeled using such labeling functions and their error rate. Although there can be some noise in such labeling, this significantly reduces the annotation cost. We sequentially apply two labeling functions, as discussed next to convert a list of medicine names to bounding boxes. In our use case of assigning a bounding box to each medicine name, we can consider it as an assignment problem between the detected bounding boxes (p) by a generic text detector and the number of medicines in it n. Considering $p = 50$ and $n = 5$, the number of possible assignments turns out to be ${}^{p}C_{n}{}^{p}P_{p} \approx 2.5e8$. We solve this problem via two techniques - using the content of the boxes (via OCR Labeling Function), and using the visual features (via Segmentation Labeling Function).

OCR Labeling Function: As for every image, we have the list of medicines that appear on it, for each detected word in the image, we can naively find the closest medicine name (by edit distance) from the ground truth list, albeit applying a threshold. However, directly using the edit distance may not respect the model's predictions. For example, according to the OCR line recognition model, modifying an i to l may have lower cost than i to z, but it would be the same edit distance for both the cases. Thus, in order to utilize the model's predictions, we decode up to the top-k predictions, and stop when we find an exact match with a medicine name from the list of ground truth medicines, i.e.,

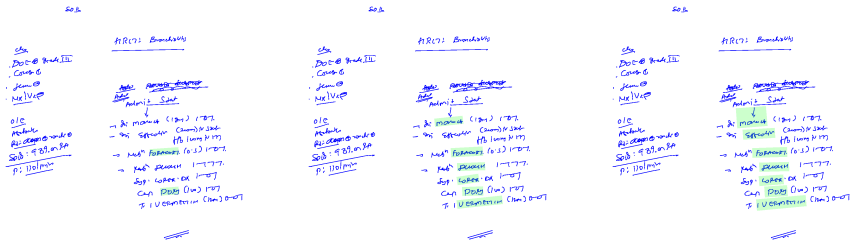
the weak labels. The bounding box associated with these matched words then can be used as the ground-truth bounding boxes of medicine names. We can define the labeling function as $\mathcal{F}(\boldsymbol{x}) = \{(t_j, l_j, h_j, w_j, r_j)\}_{j=1}^{q}$, where the bounding boxes of m medicines are in the rotated box format and t_j, l_j, h_j, w_j, r_j representing top, left, height, width, and rotation angle of each matched bounding box. Then, we can construct a training dataset as follows: $\mathcal{D}_{tr} = \{(\boldsymbol{x}_i, \mathcal{F}(\boldsymbol{x}_i))\}_{i=1}^{N}$.

The number of matching bounding boxes $q_i \leq n_i$, as in most cases the handwriting is so illegible that to decipher that even a higher number of top-k lines may not allow a match with the ground truth medicine names. This can happen for a sizable number of images, which in turn can introduce a significant noise in the data, leading to problems in learning the segmentation network. Thus, we only use those images to train our network where we find that at least 90% of the ground truth medicines have been matched. The reason behind setting such a high threshold is this set becomes the guiding signal for the rest of the algorithm. Thus our modified strongly-labeled training dataset can be represented as: $\mathcal{D}_{tr} = \{\big(\boldsymbol{x}_i, \mathcal{F}(\boldsymbol{x}_i)\big) \Big| \frac{|\mathcal{F}(\boldsymbol{x}_i)|}{|\mathcal{G}_i|} \geq 0.9\}_{i=1}^{N}$. While increasing the number of top-k paths helps more images to pass this threshold, we find that it saturates after a point, specially for documents which are hard to read, such as prescriptions used in this work. While the 0.9 threshold allows us to reduce missing bounding boxes in the training set, it also reduces the number of images in the training set, as $|\mathcal{D}_{tr}| \leq |\mathcal{D}|$. We next discuss a second labeling function to alleviate this problem.

Segmentation Labeling Function It may happen that even after decoding a high number of paths (k), we still are not able to match all the ground truth medicine names. This can happen when the handwriting is quite challenging for the model to predict accurately. In such a scenario, we leverage the visual appearance features via the segmentation model itself, rather than just labeling via OCR. Motivated by the success of self-training in domain adaptation [2, 28] and semi-supervised [7, 40], we use the segmentation model to pseudo-label the images in the rest of the dataset, i.e., $\mathcal{D}$ - $\mathcal{D}_{tr}$.

First, we train a segmentation network $\mathcal{M}$ using the relatively small training data $\mathcal{D}_{tr}$ obtained from the OCR Labeling Function outlined above. Then, we use it to predict the medicine lines on the images in $\mathcal{D}$ - $\mathcal{D}_{tr}$. We can consider the output of the model to be $\mathcal{M}(\boldsymbol{x}) = \{(t_j, l_j, h_j, w_j, r_j)\}_{j=1}^{l}$. Following our previous threshold, we add those images to the training dataset, where the union of the number of predicted medicine lines by the segmentation network and the OCR labeling function above, is at least 90% of total number of medicines in that image. We can represent the new training set as follows: $\mathcal{D}_{tr} = \{\big(\boldsymbol{x}_i, \mathcal{F}(\boldsymbol{x}_i) \cup \mathcal{M}(\boldsymbol{x}_i)\big) \Big| \frac{|\mathcal{F}(\boldsymbol{x}_i) \cup \mathcal{M}(\boldsymbol{x}_i)|}{|\mathcal{G}_i|} \geq 0.9\}_{i=1}^{N}$.

Ideally, we can repeat this process, i.e. repeat pseudo-labeling the training images using a trained segmentation model and training a new model with the pseudo-labeled training set. The training set would grow over iterations. The two labeling functions can be generalized as: $\mathcal{D}_{tr}^{T} = \{\big(\boldsymbol{x}_i, \cup_{t=1}^{T} \mathcal{M}_t(\boldsymbol{x}_i)\big) \Big| \frac{\cup_{t=1}^{T} \mathcal{M}_t(\boldsymbol{x}_i)}{|\mathcal{G}_i|} \geq$



(a) Iter 1 (b) Iter 2 (c) Iter 3

Fig. 3: Evolution of labels from the labeling functions. Iter 1 represent the OCR Labeling Function and the subsequent ones represent the Segmentation Labeling Function for different iterations. The green highlighted regions denote the detected medicine names.

$0.9\}_{i=1}^{N}$, where $\mathcal{M}_t = \mathcal{F}$ for $t = 1$, and the tth medicine line segmentation model for $t \geq 1$, and T represents the total number of iterations.

Figure 3 shows how segmentation improves over iterations. Using only the OCR Labeling Function misses out some of the medicine names, as it is dependent on the ability of the underlying OCR model we use to decipher the medicine names. However, applying the Segmentation Labeling Function on top of it helps to predict the medicine patches which were missed, as it does not depend on OCR or the content, but rather on the visual features, such as strokes, indentation, etc. which we will discuss later in Section 4.

Segmentation Model Given the bounding boxes obtained using the labeling functions, we can train a medicine line segmentation model. Our segmentation model is DeepLab [9] with a ResNet50 backbone [16]. Although we use this architecture, it can be replaced by any other state-of-the-art segmentation model. We convert the bounding boxes to label masks, and use them as supervision to train the segmentation network. The label mask has either 0 or 1 at each pixel location, denoting whether a pixel belongs to a medicine line. The segmentation model is trained with the above data using a semantic head with two output channels. The predicted medicine label masks obtained from this model may not always respect text boundaries, and hence we use a generic text detector in the OCR pipeline to detect text and refine the boundaries. Then, we crop out the detected bounding box from the original image $\boldsymbol{x}$ and send only those lines to the line recognizer. As these lines correspond to a special domain of medicine names, we can inject that knowledge to the OCR using a LM.

3.4 Domain-specific Language Model

In OCR decoder, we can incorporate a LM to correct some of the OCR errors. Specifically, the decoded string Y^* can be obtained as follows:

$$Y^* = \arg\max_{Y} P(Y|X)P(Y)^{\alpha} \tag{1}$$

where $P(Y)$ is obtained from the LM denoting the probability of occurrence of a certain string Y in the dataset, α is the weight applied on the LM, and X is the input. In a generic OCR model, $P(Y)$ is trained on a large corpus of text such that it represents a diverse set of documents. Particular to our use case, once we have detected the medicine lines as discussed in the previous section, we need only medicine line specific knowledge while decoding the OCR output. However, medicine line patterns occurring in handwritten prescriptions often do not appear in normal text. It is also difficult and expensive to acquire and annotate such large corpora of handwritten prescriptions from which we can learn medicine line specific LMs. We inject domain knowledge to solve this problem.

In order to gather medicine line specific text data, we defined a probabilistic program from which we can sample data and learn a character based LM. Medicine lines written by doctors often have a few elements - a enumeration token (-, ., numbers, etc), followed by the type of medicines (injection, tablet, etc.), the root name of the medicine, and then the suffix. These altogether comprise a single medicine name line. Note that some of these entities other than the root word may not appear in all prescriptions. With this domain knowledge, we can define a probabilistic program as shown in top-right portion of Figure 2. The program starts from the START node and ends at the END node, and concatenates the output of each node with spaces in between. To sample a medicine name line, the program takes as input the medicine name and the type of the medicine, both of which appears in the vocabulary of medicines. We can create an exhaustive set of all possible medicine name lines, and then train a character based n-gram LM on that text corpus. Note that as we do not have the exact probabilities of the different transitions, we use equi-probable transitions between nodes, as well as for any choices in the nodes.

In OCR, as decoding is done at a character level, we need character LMs, unlike recent advanced large LMs which operate on word or sub-word tokens. There are also character LM using transformers, but those are generally useful for longer context. But, in our case, medicine names on average are only 7 characters long. Moreover, using such a large model takes a lot more inference time. Hence we stick to an n-gram model.

3.5 In-Vocabulary Prediction

In many entity extraction tasks, such as medicine name prediction studied in this paper, the entities often belong to either from a fixed vocabulary, or are defined by a regular expression. However, the OCR predictions will not be constrained to our medicine vocabulary. To constrain that, we can make a nearest neighbor edit distance search for each medicine line text and the medicine vocabulary. However, as we discussed before, it would not respect the model's confidence. Thus, we use the top-k path decoding as a robust method. Specifically, for each line, we decode the top-k predictions, and then find all the text which have an exact match with one of the medicine names from the vocabulary. Then, we take a majority voting of all these matched names, and that becomes the prediction for every line. It is possible that for some of the detected medicine lines, we do not find any match

Table 1: (a) Statistics of the prescription dataset. (b) Coverage of different sections in prescriptions.

(a)

# Images	9645
# Doctors	117
Avg. medicines / image	4.5
Avg. images / doctor	82.4

(b)

Lab/Scan	70.4%
Medicine	100%
Observation	99.9%
Vital	40.5%

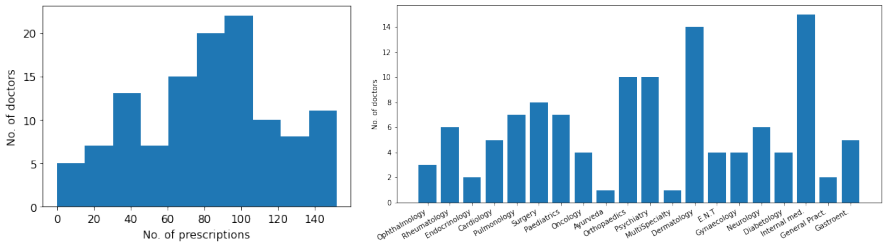
for any of the top-k prediction. These detected medicine lines would not have any output prediction. We find this method to be more effective compared to edit distance based matching with the top-1 prediction, or predicting only the first match from the top-k predictions, as shown in Section 4.

4 Experiments

We first introduce the dataset and implementation details, before sharing the experimental results and rigorous ablations to understand the efficacy of the framework.

Prescription Image Dataset: We use a dataset of handwritten prescriptions to validate the methodology outlined and evaluate the performance of the models. A few example images from the dataset are shown in Figure 1. The dataset contains 9645 images written by 117 doctors. Table 1a outlines some of the details of the dataset, and Figure 4a shows the distribution of prescription images per doctor. We use 80% of the dataset to train our models, and 20% for evaluation. There is no overlap between the doctors between the training and the test set at each iteration, ensuring that our results capture understanding across different handwriting styles. Each image in the dataset has a list of medicine names appearing in them, which we call weak labels, without any positional information. However, just for evaluation, we strongly annotate 500 images from the evaluation set to evaluate the segmentation performance. Prescriptions generally have multiple other sections as well (although unstructured in free-form), and Table 1b shows the percentage of images which have other sections such as lab/scans reported, observations and vitals. Also, note that any and all personally identifiable information was removed from the data prior to it being provided to the authors for this study.

Medicine Vocabulary: We also use a medicine name vocabulary consisting of more than 90,000 medicine names. We use this to generate synthetic medicine name lines and train the character based medicine LM. This vocabulary is also used to make the in-vocabulary predictions.



(a) (b)

Fig. 4: (a) This plot shows the number of prescriptions per doctor in the dataset, (b) This plot shows the number of doctors per specialty.

Evaluation Protocol: We evaluate all models on test set of the dataset mentioned above. To evaluate the performance of the segmentation model, we use mean intersection over union (mIoU) as used in the segmentation literature [8]. To evaluate the performance of the end-to-end medicine name prediction model, we use the mean jaccard index, over all the images. We also use two other metrics namely mean precision and mean recall, and the mean jaccard index can be considered as a combination of both these metrics. These are defined as follows -

$$\text{Mean Jaccard Index (mJI)} = \frac{1}{M}\sum_{i=1}^{M}\frac{|P_i \cap G_i|}{|P_i \cup G_i|} \tag{2}$$

$$\text{Mean Precision (mP)} = \frac{1}{M}\sum_{i=1}^{M}\frac{|P_i \cap G_i|}{|P_i|}, \tag{3}$$

$$\text{Mean Recall (mR)} = \frac{1}{M}\sum_{i=1}^{M}\frac{|P_i \cap G_i|}{|G_i|} \tag{4}$$

where P_i, G_i are the predicted and ground truth list of medicines for the ith image. M is the number of evaluation images. The comparison between the prediction and ground-truths are not case-sensitive, as they are medicine names.

4.1 Results and Ablation Studies

Iterative Training Performance: As discussed in Section 3, our algorithm for converting weak labels (only medicine names) to strong labels (bounding box annotations for each medicine name) involves two labeling functions - OCR and Segmentation Labeling Function, where the latter can be applied iteratively. The number of images auto-labeled by the labeling functions increases with iterations, and hence the performance of both the medicine line segmentation model as well as the medicine name prediction model increases with subsequent iterations. We highlight this in Table 2. Iteration 1 shows the performance on only OCR Labeling Function, and Iteration ≥ 2 shows the performance on multiple iterations of

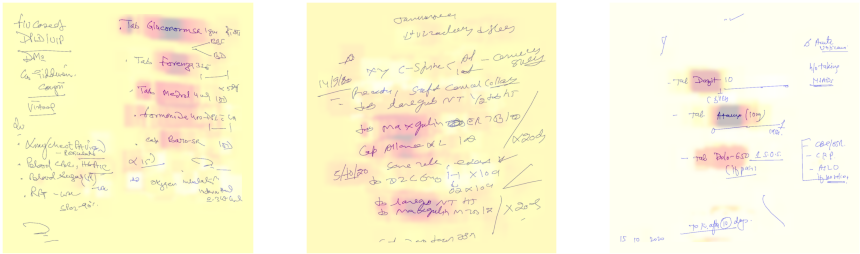
Table 2: Performance over iterations of the proposed framework. Iter 1 represents learning from only the OCR Labeling Function and iter ≥ 2 shows the performance after iteratively including the Segmentation Labeling Function. The medicine name performances are only for topk=1. GT bbox shows the performance when the groundtruth bounding boxes are provided for medicine names only during evaluation.

Iteration	1	2	3	GT bbox
Train data (%)	24.4	66.3	90.2	-
Segmentation (mIoU)	72.6	77.9	77.2	100%
Medicine Name (mJI)	44.8	45.9	45.9	49.8%

Segmentation Labeling Function. For a significant number of prescriptions, it is difficult to decipher some of the medicine names, even when we use a high value of top-k (k=20,000 in our experiments) decoded outputs per line. For Iteration 1, the number of auto-labeled prescriptions is $< 25\%$ of the training set. This shows the difficulty level of the problem at hand. Note that the train sets are used to train only the medicine line segmentation model and not the lines recognizer of the OCR, thus it can be with any off-the-shelf OCR model.

The segmentation performance as well as the medicine name performance improve over iterations but saturates from Iteration 3. Note that mIoU computes the performance for every pixel, but normally a small change in the final bounding box do not have a lot of impact on the medicine name prediction, as long as they encapsulate the text within it. We also show the upper bound performance of medicine line recognition by using ground-truth medicine bounding boxes only while evaluating. As we can see, our algorithm with just using weak labels can reach within a few points of the strongest upper-bound with strong labels.

Cues for medicine name segmentation: Unlike a generic text detector, specifically detecting medicine lines can be challenging, as handwritten prescriptions do not have any specific structure or location in the page. However, the segmentation model is still able to predict the location of the medicine lines with high performance as shown in Table 2. In order to understand the cues the segmentation model uses to segment the medicine names, we do the following experiment. Given a test image $\boldsymbol{x}$, using a sliding window, we remove square patches from the image to remove potential cues, one at a time. Consider $\boldsymbol{x}_{i,j}$ as the image when patch at location (i, j) is removed. We can run the segmentation model on this image, $\mathcal{M}(\boldsymbol{x}_{i,j})$ and obtain the mIoU. For every location (i, j), on the image, we can obtain the model's performance drop when a patch around that is removed, and then display that as a heatmap. A drop in performance in certain regions of this image depicts the regions necessary for the segmentation model to segment the medicine names correctly. As we can see in Fig. 5, the model is clearly utilizing cues from visual features surrounding medicine lines such as starting of a line like Tab, Cap, hyphens, etc. These observations are aligned



(a) (b) (c)

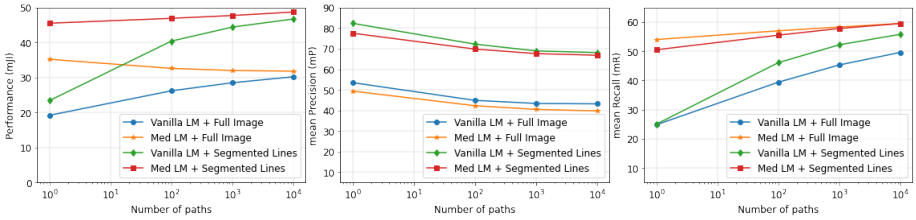
Fig. 5: Cues needed by the segmentation network. Deeper color denotes lower performance when a patch around that is removed. A few parts of the image other than the medicine names, such as hyphens, Tab, Cap, etc., also appear to be darker, which are some of the cues that the model looks at to determine whether it is a medicine line.

with what a pharmacist or even non-domain experts look to determine medicine lines, as in most cases the handwriting is illegible. These key demarcations serve as strong signals to recognize medicine lines, after which we can condition our knowledge to medicine names to enhance line recognition.

Contribution of medicine LM and segmentation model: Here we show how selectively injecting medicine LMs can offer a significant improvment in performance. The vanilla LM is trained on a generic corpus of text from the Latin script. However, the medicine name LM is trained as discussed in Section 3.4. The performance improves with path length for both the models but for the medicine LM, the top-1 path itself performs much better than top-1000 path for the vanilla LM (Fig. 6). This also reduces the compute time in decoding the top-k paths from the logits, which is linear in the number of paths.

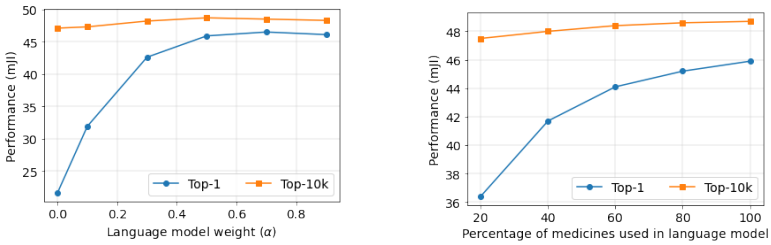
Moreover, segmenting and selectively injecting the LM plays a critical role on the performance, and MedLM + Segmented Lines perform the best. Applying the MedLM on the full image actually reduces the precision significantly, but improves the recall slightly as expected, but reducing the overall metric, i.e., jaccard index. This shows that selectively injecting the LM is important, otherwise it can mess up the rest of the prescription, and hallucinate medicine names from them.

Performance with varying weight on LM: The weight α in Eqn 1 on the LM scores can have an impact on the final performance. A low weight may lead to no improvement beyond the optical model's prediction, and a high weight may not ground the output to the actual text on the image. Figure 7a shows an ablation of the medicine name prediction performance on the LM weight. Note that the changes in performance is much lower for top-10k paths than for top-1 path, as only the first path in the top-10k path is affected by the LM because for paths > 1, the predictions come from the top-k decoded paths which is based on only the logits without any LM scoring. Nonetheless, we see that the performance of both the models are very close after a certain value of α.



(a) Jaccard Index (b) Precision (c) Recall

Fig. 6: Jaccard index, precision and recall comparison using different language models and inputs (medicine line segmented and full page). The medicine LM on segmented medicine lines works the best, the top-1 of which is better than the top-1000 of the vanilla LM. Applying the medicine LM on the entire image decreases the precision of the predictions, as it hallucinates medicine names in the rest of the prescription.



(a) (b)

Fig. 7: (a) Ablation of performance with weight on the language model α. $\alpha = 0$ denote the performance of only the optical model. (b) Ablation of fraction of medicine names used to train the medicine language model. We present the performance when top-1 and top-10k paths are used to predict after vocabulary matching.

Varying the vocabulary of the LM: The medicine names used in generating the synthetic lines can have an impact on the quality of the medicine name LM. Here we also show how the performance varies as we increase the number of medicine names used to train the medicine LM. Figure 7b presents the results for top-1 and top-10k with different size of medicine name dataset. The performance improves as we add more medicines, but starts saturating after a certain point.

Performance with different n-gram models: The n-gram LM involves a parameter n, which is the number of history characters the model looks to obtain the score of the next character. We created multiple n-gram models on the synthetically generated medicine line text data, and show the results in Table 3. More context definitely helps in performance, but it saturates after $n = 7$. This is also intuitive as the length of the medicine names is around 7.9 on average.

Predicting In-Vocabulary Words: In the final step of our algorithm to predict medicine names, we only predict those words where we find a direct match with

Table 3: Ablation of different n-gram models trained on medicine line data.

	n=3	n=5	n=7	n=9
Top-1 (mJI) (%)	27.2	41.5	45.9	45.9
Top-10k (mJI) (%)	47.4	48.1	48.7	48.7

Table 4: Ablation of different algorithms to predict medicine names. We use $k = 1e4$.

	top-1	top-1-edit	top-k	top-k+majority
Jaccard Index	45.9	45.8	46.9	48.7
Precision	76.9	68.6	64.8	66.8
Recall	51.0	54.4	58.5	59.5

one of the elements of the medicine vocabulary. As discussed before, finding a match for only the top-1 prediction may not be the best. Thus, we decode until top-k and find matches for all the text. As the top-k decoding is directly dependent on the output of the model, such a matching respects the model's predictions. We then take a majority voting of all the matches and that becomes the final predicted medicine for a line. Note that some lines may not have any prediction at all. In this section, we compare multiple strategies of predicting in-vocabulary words in Table 4. Top-1 represents an exact match with the first path, top-1 edit distance finds the nearest prediction from the vocabulary by edit distance, top-k denotes we decode the top-k outputs but stop when we find the first exact match, and finally top-k+majority is the algorithm we use, where we decode all the top-k lines and take a majority voting of all the exact matches.

Note that top-1-edit has the same jaccard index as top1, but the former has lower precision with higher recall than top-1, as expected, because it predicts beyond exact matches. We tried with multiple thresholds for edit distance, and found that 85% normalized distance performs the best. Increasing the threshold, i.e., allowing more matches significantly reduces the precision, at the gain of the recall, but hurting the overall performance. This is because of the intuition we discussed earlier that topk decodings respect the model's confidence, but edit distance treats every replacement with the same cost.

4.2 Error-mode analysis

The two types of errors possible are - medicine names predicted but not in the ground-truth (type I) and medicine names in the ground-truth but not predicted (type II). In our framework, there are two reasons behind the errors - segmentation network and OCR. If a medicine name is not segmented, then it leads to a type-II error. OCR errors contributes to the rest (type I and type II), a majority of which is contributed by misinterpreting very similar looking medicines such as

emtel vs entel, eenosol vs eenasof, folvite vs folite, paro vs baro, zincovit vs zincort, aloliv vs alcoliv. Also we observe that the doctor can commit spelling mistakes, or vaguely write a medicine name, where only the first few characters are recognizable. To correct such errors, pharmacists generally use other contexts such as observation. Learning such contexts would need a lot more data, and injecting higher-level domain knowledge.

5 Conclusion

In this paper, we looked into the problem of extracting medicine names from inscrutable handwritten prescriptions. Our algorithm can selectively infuse domain knowledge to specific portions of a document to significantly improve the performance. We developed a framework that can learn to detect regions of interest from just weak labels, and also learn a medicine language model using synthetically generated text lines using probabilistic programs. The idea is generic enough to be applied to a variety of other types of documents, such as handwritten forms.

Acknowledgement: We thank Srujana Merugu, Ansh Khurana, Manish Gupta, Harsh Dhand and Shruti Garg for all the support and discussions during the course of this project. Without their effort, this project would not have been possible.

References

1. Achkar, R., Ghayad, K., Haidar, R., Saleh, S., Al Hajj, R.: Medical handwritten prescription recognition using crnn. In: CITS. IEEE (2019)
2. Araslanov, N., Roth, S.: Self-supervised augmentation consistency for adapting semantic segmentation. In: CVPR (2021)
3. Bhunia, A.K., Sain, A., Chowdhury, P.N., Song, Y.Z.: Text is text, no matter what: Unifying text recognition using knowledge distillation. In: ICCV (2021)
4. Bissacco, A., Cummins, M., Netzer, Y., Neven, H.: Photoocr: Reading text in uncontrolled conditions. In: ICCV. pp. 785–792 (2013)
5. Breuel, T.M., Ul-Hasan, A., Al-Azawi, M.A., Shafait, F.: High-performance ocr for printed english and fraktur using lstm networks. In: ICDAR. IEEE (2013)
6. Bukhari, S.S., Kadi, A., Jouneh, M.A., Mir, F.M., Dengel, A.: anyocr: An open-source ocr system for historical archives. In: ICDAR (2017)
7. Cascante-Bonilla, P., Tan, F., Qi, Y., Ordonez, V.: Curriculum labeling: Revisiting pseudo-labeling for semi-supervised learning. In: AAAI (2021)
8. Chen, L.C., Papandreou, G., Kokkinos, I., Murphy, K., Yuille, A.L.: Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. PAMI (2017)
9. Cheng, B., Collins, M.D., Zhu, Y., Liu, T., Huang, T.S., Adam, H., Chen, L.C.: Panoptic-deeplab: A simple, strong, and fast baseline for bottom-up panoptic segmentation. In: CVPR. pp. 12475–12485 (2020)

10. Diaz, D.H., Qin, S., Ingle, R., Fujii, Y., Bissacco, A.: Rethinking text line recognition models. arXiv preprint arXiv:2104.07787 (2021)
11. D'hondt, E., Grouin, C., Grau, B.: Generating a training corpus for ocr post-correction using encoder-decoder model. In: IJCNLP (2017)
12. Fujii, Y., Driesen, K., Baccash, J., Hurst, A., Popat, A.C.: Sequence-to-label script identification for multilingual ocr. In: ICDAR. IEEE (2017)
13. Graves, A., Fernández, S., Gomez, F., Schmidhuber, J.: Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In: Proceedings of the 23rd international conference on Machine learning. pp. 369–376 (2006)
14. Gupta, H., Del Corro, L., Broscheit, S., Hoffart, J., Brenner, E.: Unsupervised multi-view post-ocr error correction with language models. In: EMNLP. pp. 8647–8652 (2021)
15. Gupta, M., Soeny, K.: Algorithms for rapid digitalization of prescriptions. Visual Informatics (2021)
16. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: CVPr. pp. 770–778 (2016)
17. Huang, J., Pang, G., Kovvuri, R., Toh, M., Liang, K.J., Krishnan, P., Yin, X., Hassner, T.: A multiplexed network for end-to-end, multilingual ocr. In: CVPR (2021)
18. Ingle, R.R., Fujii, Y., Deselaers, T., Baccash, J., Popat, A.C.: A scalable handwritten text recognition system. In: ICDAR (2019)
19. Jayakumar, P.: Online doctor consultation market to grow (2021), https://www.businesstoday.in/lifestyle/health/story/online-doctor-consultation-market-to-grow-72-to-836-million-by-march-2024-study-304689-2021-08-1
20. Karatzas, D., Gomez-Bigorda, L., Nicolaou, A., Ghosh, S., Bagdanov, A., Iwamura, M., Matas, J., Neumann, L., Chandrasekhar, V.R., Lu, S., et al.: Icdar 2015 competition on robust reading. In: ICDAR. IEEE (2015)
21. Karthikeyan, S., de Herrera, A.G.S., Doctor, F., Mirza, A.: An ocr post-correction approach using deep learning for processing medical reports. TCSVT (2021)
22. Khoreva, A., Benenson, R., Hosang, J., Hein, M., Schiele, B.: Simple does it: Weakly supervised instance and semantic segmentation. In: CVPR (2017)
23. Kittenplon, Y., Lavi, I., Fogel, S., Bar, Y., Manmatha, R., Perona, P.: Towards weakly-supervised text spotting using a multi-task transformer. In: CVPR (2022)
24. Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C.H., Kang, J.: Biobert: a pre-trained biomedical language representation model for biomedical text mining. bioinformatics btz682 (2019)
25. Li, D., Huang, J.B., Li, Y., Wang, S., Yang, M.H.: Weakly supervised object localization with progressive domain adaptation. In: CVPR (2016)
26. Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., Wei, F.: Trocr: Transformer-based optical character recognition with pre-trained models. arXiv preprint arXiv:2109.10282 (2021)
27. Litman, R., Anschel, O., Tsiper, S., Litman, R., Mazor, S., Manmatha, R.: Scatter: selective context attentional scene text recognizer. In: CVPR (2020)
28. Liu, H., Wang, J., Long, M.: Cycle self-training for domain adaptation. NeurIPS (2021)
29. Long, S., He, X., Yao, C.: Scene text detection and recognition: The deep learning era. IJCV (2021)
30. Long, S., Qin, S., Panteleev, D., Bissacco, A., Fujii, Y., Raptis, M.: Towards end-to-end unified scene text detection and layout analysis. In: CVPR (2022)

31. Marti, U.V., Bunke, H.: The iam-database: an english sentence database for offline handwriting recognition. IJDAR (2002)
32. Paul, S., Roy, S., Roy-Chowdhury, A.K.: W-talc: Weakly-supervised temporal activity localization and classification. In: ECCV (2018)
33. Pragnadyuti, M., Rabindranath, D., Suhrita, P., Kumar, S.A., Kumar, J.S.: Legibility assessment of handwritten opd prescriptions of a tertiary care medical college and hospital in eastern india. SJMPS (2017)
34. Rani, S., Rehman, A.U., Yousaf, B., Rauf, H.T., Nasr, E.A., Kadry, S.: Recognition of handwritten medical prescription using signature verification techniques. CMMM (2022)
35. Rasmy, L., Xiang, Y., Xie, Z., Tao, C., Zhi, D.: Med-bert: pretrained contextualized embeddings on large-scale structured electronic health records for disease prediction. Nature (2021)
36. Ratner, A., Bach, S.H., Ehrenberg, H., Fries, J., Wu, S., Ré, C.: Snorkel: Rapid training data creation with weak supervision. In: VLDB. NIH Public Access (2017)
37. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In: CVPR (2016)
38. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: Towards real-time object detection with region proposal networks. NeurIPS (2015)
39. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: CVPR (2018)
40. Sohn, K., Berthelot, D., Carlini, N., Zhang, Z., Zhang, H., Raffel, C.A., Cubuk, E.D., Kurakin, A., Li, C.L.: Fixmatch: Simplifying semi-supervised learning with consistency and confidence. NeurIPS (2020)
41. Thompson, P., McNaught, J., Ananiadou, S.: Customised ocr correction for historical medical text. In: 2015 digital heritage. IEEE (2015)
42. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. NeurIPS (2017)
43. Wang, P., Li, H., Shen, C.: Towards end-to-end text spotting in natural scenes. PAMI (2021)
44. Wei, Y., Liang, X., Chen, Y., Shen, X., Cheng, M.M., Feng, J., Zhao, Y., Yan, S.: Stc: A simple to complex framework for weakly-supervised semantic segmentation. PAMI (2016)
45. Yang, X., PourNejatian, N., Shin, H.C., Smith, K.E., Parisien, C., Compas, C., Martin, C., Flores, M.G., Zhang, Y., Magoc, T., et al.: Gatortron: A large clinical language model to unlock patient information from unstructured electronic health records. arXiv preprint arXiv:2203.03540 (2022)
46. Zhang, C., Cao, M., Yang, D., Chen, J., Zou, Y.: Cola: Weakly-supervised temporal action localization with snippet contrastive learning. In: CVPR (2021)
47. Zhang, D., Han, J., Cheng, G., Yang, M.H.: Weakly supervised object localization and detection: A survey. PAMI (2021)